

Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #3
List operations + The Cut
Computer Science

Agenda

- **Introducing recursion types**
- **Example – length of a list**
 - Types of parameters
- **Example – reversing a list**
 - Introducing pattern position
 - Meaning of arguments
- **Backtracking prevention – the CUT (!)**
 - Negation (as failure)

Recursion types

- What does recursion require?
 - Call to itself
 - Stopping condition
 - Step (*to get to the stopping condition*)
 - Process (*to get the result?*)

- **Backward**

```
bw([], ?) .
```

```
bw([H|T], R) :-
```

```
    bw(T, Rpart),  
    R created from Rpart
```

- **Differences?**

- When and where the result is constructed. **Backward** recursion builds the result as recursion returns, while **forward** builds a partial result on each recursive call.

- **Forward**

```
fw([], ?) .
```

```
fw([H|T], R) :-
```

```
    NewR created from R  
    fw(T, NewR) .
```

Actually, not quite like this, we shall see why in the following slides.

List length - backward recursion

- Determine the length of a list, number of arguments?

```
% len_bw/2 (+List, -Length)
```

```
len_bw([],0).
```

% length of empty list is 0

```
len_bw([_|T],N):-
```

```
    len_bw(T,N1),
```

% length of tail calculated as N1

```
    N is N1+1.
```

% add 1 to the length of tail

- Order of clauses? This way? Swapped? Which is correct? Which is better? Why? Is it important? Why?
 - (discuss indexation on the first argument. Oral explanation.)
- Meaning of N1?
 - partial result (N1) not quite relevant (is length of the tail)
- What if we need to keep the number of “consumed” elements?
- This solution uses **backward** recursion, meaning that the **result is built as recursion RETURNS**

List length - forward recursion

- What if we build it while advancing?

```
% len_fw/2 (+List, -Length)
```

```
len_fw(PR, [_|T]) :-
```

```
    NPR is PR+1,
```

% add 1 to the partial result (PR) so far
% i.e. length of the covered part

```
    len_fw(NPR, T) .
```

% go count the rest of the elements

```
len_fw(R, []) .
```

% R has the final list length

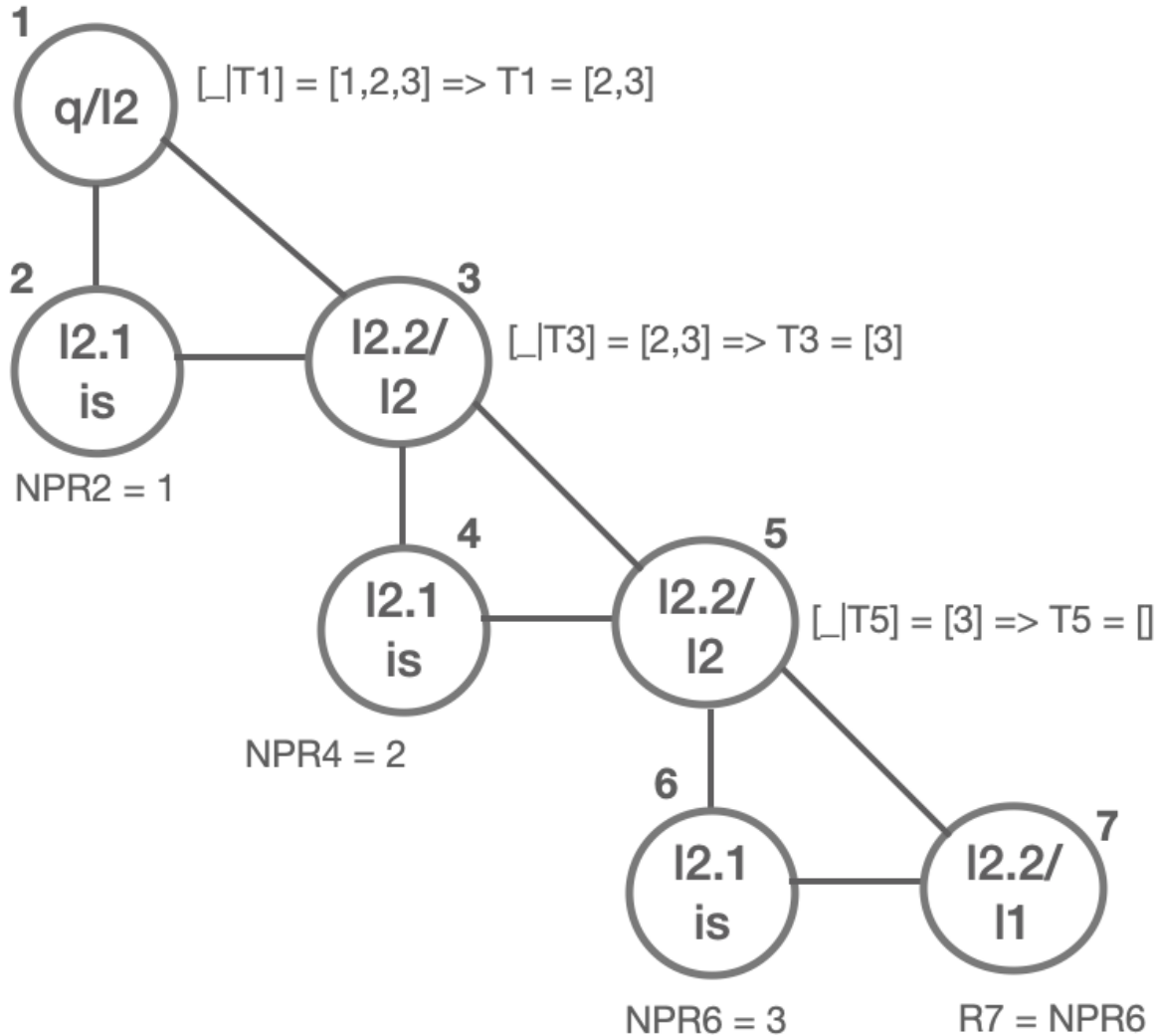
- What's the difference compared to the previous solution?
- PR represents here what we *have covered*; before – what we've *yet to cover*.

Computer Science

len_fw/2

Deduction tree

```
[11] len_fw([],R).
[12] len_fw([_|T],PR):-
[121]   NPR is PR+1,
[122]   len_fw(T, NPR).
[q] ?- len_fw([1,2,3],?).
```



List length - forward recursion

```
% len_fw/2 (+List, -Length)
```

- Not enough – as result, although calculated, is NEVER reported

```
len_fw([],N) .
```

```
len_fw([_|T],N) :-
```

```
    N is N1+1,
```

```
    len_fw(T,N1) .
```

- Upgrade it with 3rd argument = “steal” the result from the rightmost leaf

```
len_fw([],PR,FR) :-FR=PR.
```

```
len_fw([_|T],PR,FR) :-
```

```
    NPR is PR+1,
```

```
    len_fw(T,NPR,FR) .
```

- Observe that in the second clause does not modify FR at all

List length – forward recursion

- With default unification:

```
len_fw([],R,R) .  
len_fw([_|T],PR,FR) :-  
    NPR is PR+1,  
    len_fw(T,NPR,FR) .
```

- How should we call it? Needs specific INITIALIZATION:

```
?- len_fw(InList,0,Result) .
```

- To avoid mandatory initial call, use a wrapper:

```
len_fw(InList,Result) :-  
    len_fw(InList,0,Result) .
```

- Can have same name? Why/why not?
- Thus, the query becomes:

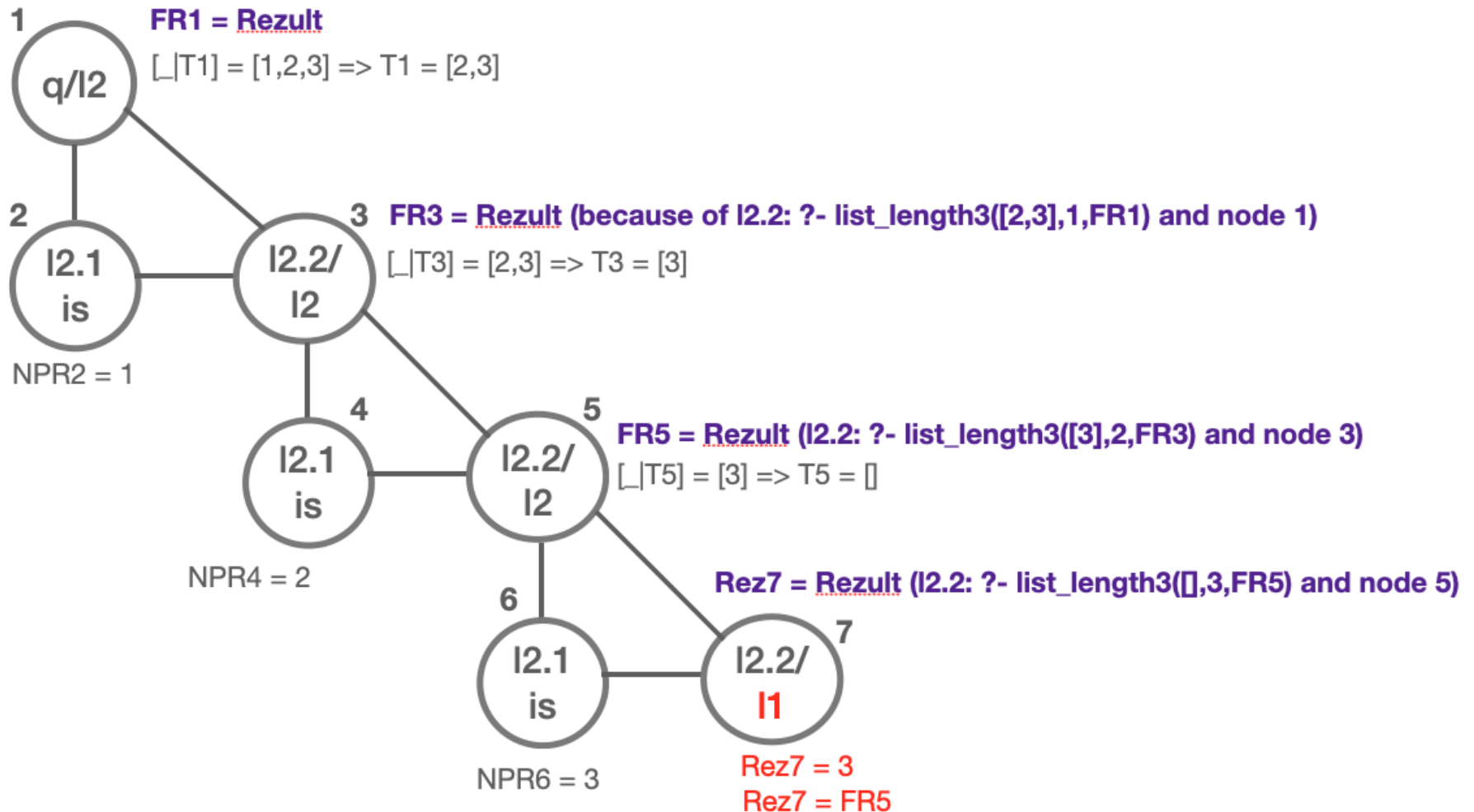
```
?- len_fw(InList,Result) .
```


deduction tree

```
[11] len_fw([],Res,Res).
[12] len_fw([_|T],PR,FR):-
[121]     NPR is PR+1,
[122]     len_fw(T,NPR,FR).
```

```
[q] ?- len_fw(InList,0,Result).
```

```
[q] ?- len_fw(InList,Result). %wrapper
```



Forward vs Backward recursion

Forward recursion

+

- Potentially useful partial results
- Efficiency if parallelized
- LCO (Last Call Optimization)

-

- Need for extra variable (final result)
- Need for specialized call (wrapper for accumulator initialization)

Backward recursion

- No need for extra variable
- No need for special call (*wrapper*)

- Rarely useful intermediate results
- Not efficient if parallelized (execution of concurrent process(es) - delayed until recursion returns)

List reverse - backward recursion

- How many arguments?

- 2 = list + reversed list

```
% rev_bw/2 (+List, -RevList)
```

```
rev_bw([], []). % empty input returns empty output
```

```
rev_bw([H|T], R) :-
```

```
    rev_bw(T, PR), % reverses the tail of list
```

```
    append(PR, [H], R). % concatenates the partial results
```

- Needs specialized call? Why/why not?
- Efficiency? How is it calculated?
- Can we avoid 2 traversals through the list? How?

Computer Science

List reverse - forward recursion

- How many arguments?

```
% rev_fw/3(+List, +PartRevList, -RevList)
```

```
rev_fw([], PR, PR) .
```

% when input gets empty the partial
% result becomes the final one

```
rev_fw([H|T], PR, R) :-  
    NPR=[H|PR],
```

% concatenates the partial results
% PR, NPR acts as a stack

```
rev_fw(T, NPR, R) .
```

% reverses the tail of list

- Wrapper (MUST have)

```
rev_fw(In, Out) :-  
    rev_fw(In, [], Out) .
```

- Efficiency? Is it better?

List reverse - forward recursion

```
reverse([], PR, PR) .  
reverse([H|T], PR, R) :-  
    reverse(T, [H|PR], R) .
```

- Same as before, but explicit → implicit unification
- Let's change the order of arguments 2 and 3 (makes no difference in the functionality)

```
reverse([], PR, PR) .  
reverse([H|T], R, PR) :-  
    reverse(T, R, [H|PR]) .
```

- Rename variables:

```
reverse([], L, L) .  
reverse([H|T], L, R) :-  
    reverse(T, L, [H|R]) .
```

Comparative analysis

```
reverse([], L, L) .
reverse([H|T], L, R) :-
    reverse(T, L, [H|R]) .

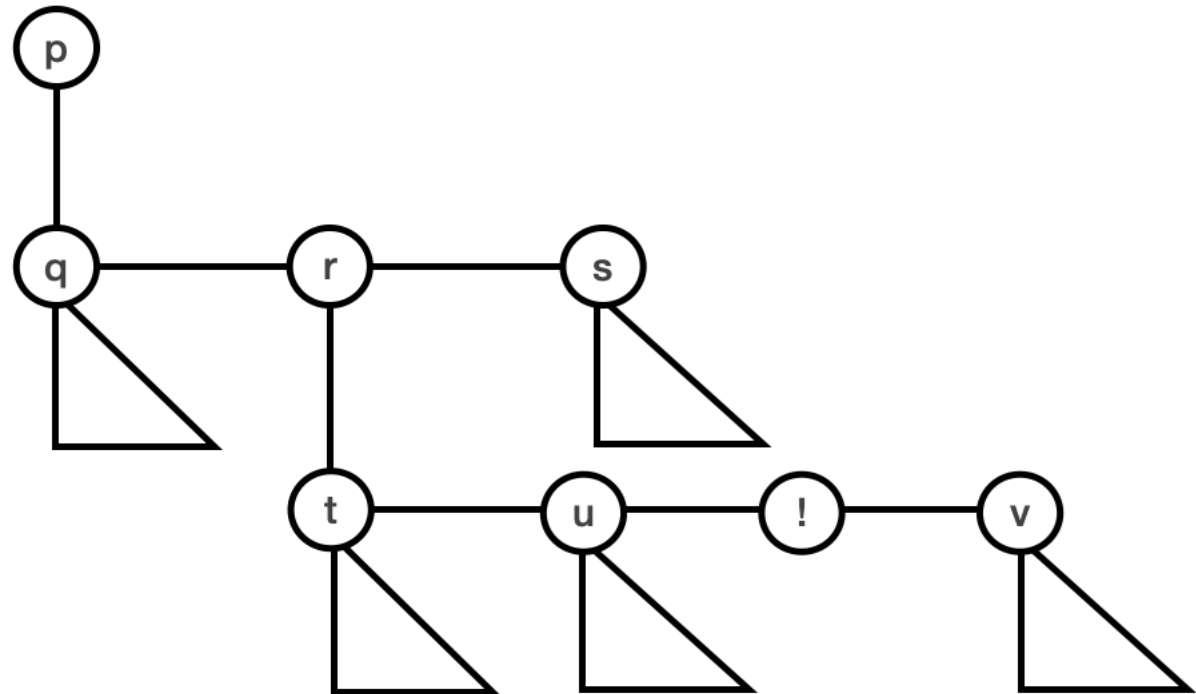
append([], L, L) .
append([H|T], L, [H|R]) :-
    append(T, L, R) .
```

- What is the difference?
- Meaning of pattern's position
- Length of arguments analysis

Backtracking avoidance (the CUT !)

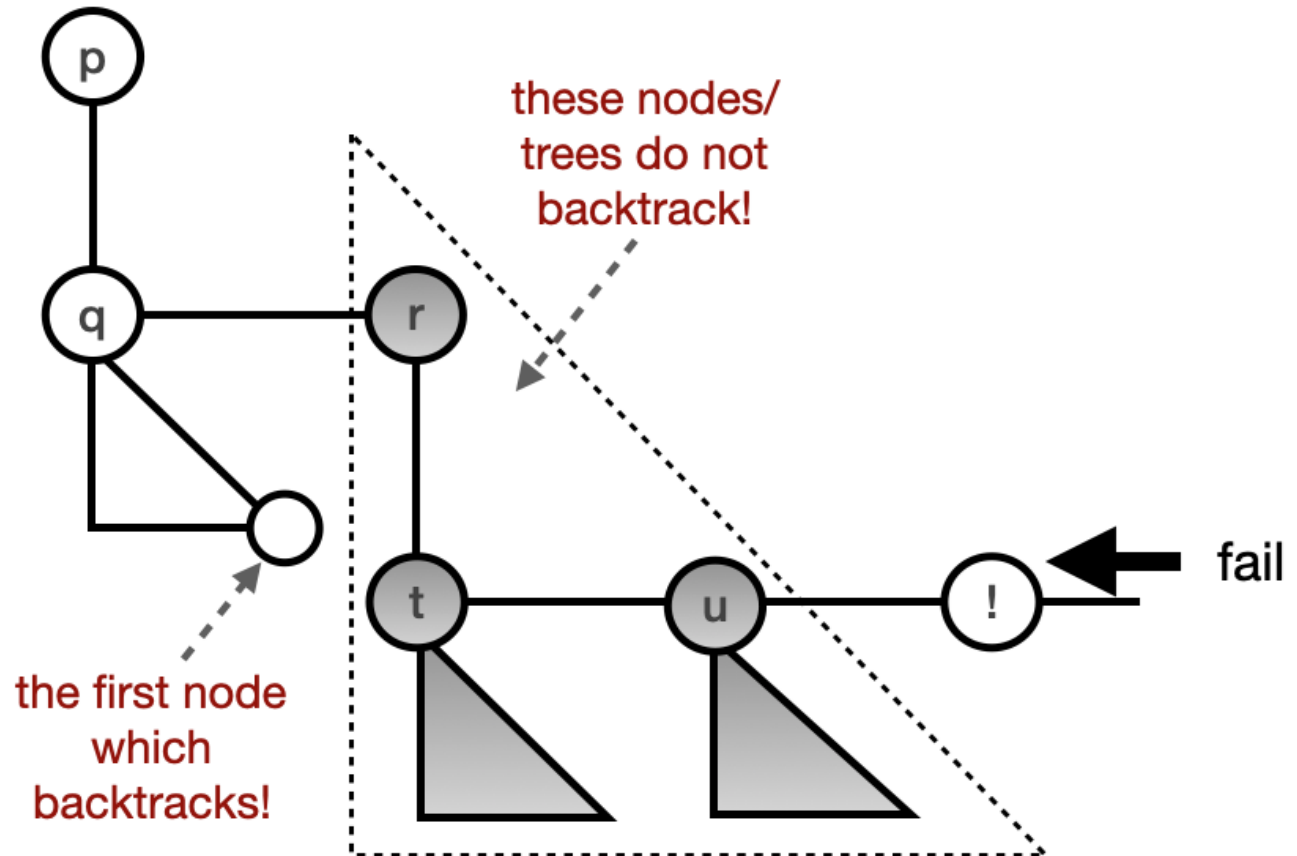
- Built-in predicate, the “!” operator
- Always succeeds
- Never backtracks

$p:-q, r, s.$
 $r:-t, u, !, v.$



The CUT (!)

$p:-q,r,s.$
 $r:-t,u,! ,v.$



The CUT (!) – effects

- **!** Prevents the backtracking of the following categories:
 - All left siblings
 - All subtrees of left sibling nodes
 - Parent node
- NO other node is affected:
 - Siblings on the right/their subtrees have the ability to backtrack
 - Left siblings DO backtrack BEFORE **!** Is executed
 - Their subtrees also
- As everything, **!** takes effect ONLY AFTER its execution (before execution it does NOT actually exist)

Negation (as failure)

- Unfair behavior
- As it assumes the **inability to prove True means False**

`not (P) :-`

`P, !, fail.`

`not (P) .`

- $P - \text{true} \Rightarrow \text{not}(P) - \text{false}$; OK.
- $P - \text{false} \Rightarrow \text{not}(P) - \text{true}$; OK.
- $P - ?$ Don't know how it is. Why? Incomplete information, not known at the time.
- How is `not(P)`? Let's run it:
 - P 's execution fails (P is NOT yet True),
 - we enter the second clause and succeeds.

Negation (as failure)

```
not (P) :-  
    P, !, fail.  
not (P) .
```

- Negation - assumes negated is True just because we don't have information about P.
- Negation as failure (to check validity)